

The numeric and reflected dunder methods always create new objects rather than modifying the object in-place. The *in-place dunder methods*, called by the augmented assignment operators, such as `+=` and `*=`, modify the object in-place rather than creating new objects. (There is an exception to this, which I'll explain at the end of this section.) These dunder method names begin with an *i*, such as `__iadd__()` and `__imul__()` for the `+=` and `*=` operators, respectively.

For example, when Python runs the code `purse *= 2`, the expected behavior isn't that the `WizCoin` class's `__imul__()` method creates and returns a new `WizCoin` object with twice as many coins, and then assigns it the `purse` variable. Instead, the `__imul__()` method modifies the existing `WizCoin` object in `purse` so it has twice as many coins. This is a subtle but important difference if you want your classes to overload the augmented assignment operators.

Our `WizCoin` objects already overload the `+` and `*` operators, so let's define the `__iadd__()` and `__imul__()` dunder methods so they overload the `+=` and `*=` operators as well. In the expressions `purse += tipJar` and `purse *= 2`, we call the `__iadd__()` and `__imul__()` methods, respectively, with `tipJar` and `2` passed for the `other` parameter, respectively. Add the following to the end of the `wizcoin.py` file:

```
--snip--
def __iadd__(self, other):
    """Add the amounts in another WizCoin object to this object."""
    if not isinstance(other, WizCoin):
        return NotImplemented

    # We modify the `self` object in-place:
    self.galleons += other.galleons
    self.sickles += other.sickles
    self.knuts += other.knuts
    return self # In-place dunder methods almost always return self.

def __imul__(self, other):
    """Multiply the amount of galleons, sickles, and knuts in this object
    by a non-negative integer amount."""
    if not isinstance(other, int):
        return NotImplemented
    if other < 0:
```